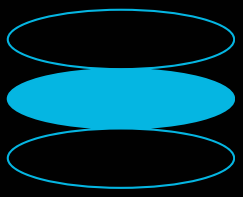
LIST CHAINéE & DOUBLEMENT CHAINéENT



1. Liste chaînée (Linked List) :

- Définition :

Une **liste chaînée** est une **collection d'éléments (nœuds)** où chaque élément contient **des données** et un **pointeur vers l'élément suivant.**

- Explication:

- ☐ La mémoire est **allouée dynamiquement** pour chaque nœud.
- ☐ Permet **ajout ou suppression d'éléments facilement** sans déplacer tout le tableau.

- Exemple :

```
#include <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node *next;

};

int main() {

    struct Node *head = NULL;
```

```
    struct Node *second = NULL;
    struct Node *third = NULL;
    // allocation mémoire
    head = (struct Node*) malloc(sizeof(struct Node));
    second = (struct Node*) malloc(sizeof(struct Node));
    third = (struct Node*) malloc(sizeof(struct Node));
    // assigner les valeurs
    head->data = 10; head->next = second;
    second->data = 20; second->next = third;
    third->data = 30; third->next = NULL;
    // afficher
    struct Node *ptr = head;
    while(ptr != NULL) {
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
    return 0;
}
```

2. Liste doublement chaînée (Doubly Linked List):

- Définition :

Une **liste doublement chaînée** est une **liste chaînée avec deux pointeurs par nœud** :

- next → élément suivant
- prev → élément précédent

• Explication:

- Permet **parcourir la liste dans les deux directions**.
- Plus flexible pour les insertions et suppressions.

• Exemple :

```
#include <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node *next;

    struct Node *prev;

};

int main() {

    struct Node *head = NULL;

    struct Node *second = NULL;

    head = (struct Node*) malloc(sizeof(struct Node));

    second = (struct Node*) malloc(sizeof(struct Node));

    head->data = 10; head->prev = NULL; head->next = second;

    second->data = 20; second->prev = head; second->next = NULL;

    // afficher de début à fin

    struct Node *ptr = head;

    while(ptr != NULL) {

        printf("%d -> ", ptr->data);

        ptr = ptr->next;
```

```
    }
    printf("NULL\n");
    return 0;
}
```